AI-Guided Prioritization of Protein Mutations for Stability and Functional Retention

Medvolt.AI — AI Research Scientist case study
Dataset: `Protein_Mutation_Assignment_Dataset` (fully synthetic)
Random seed: `20261102` · Environment: Python 3.11.15, scikit-learn 1.9.1, numpy 2.4.6, torch 2.15.0
Code: `medvolt_protein_engineering/` · 807 automated tests, `ruff` and `mypy` clean

1. Problem and scientific objective

Protein engineering campaigns face a combinatorial search problem. A 300-residue protein admits 5,700 single-point substitutions; a wet-lab campaign can characterise tens. The binding constraint is not ideas but assay throughput, so the practical question is not "can we predict mutation effects" but "which handful of mutations should enter the queue next".

That question has two objectives that trade off. A mutation must be **thermodynamically tolerated** — it must not unfold the protein — and it must **retain function**. Optimising either alone fails: the most stabilizing substitutions are frequently at buried positions where they disturb an active site, while the most functionally conservative changes are often thermodynamically neutral and therefore uninteresting. A useful prioritization must be explicitly multi-objective.

This study builds an end-to-end system that predicts `stability_ddG` (negative = stabilizing) and `functional_score` (0 = severe loss, 1 = wild-type-like) for unlabeled mutations; estimates **per-candidate uncertainty**; flags candidates outside the training distribution; and returns a ranked, justified Top 10. The emphasis throughout is on **honest performance estimation**. For a system whose output decides which molecules get synthesized, an over-optimistic validation score is worse than a pessimistic one: it silently converts a modelling artefact into wasted bench time.
Section 4 shows the obvious validation choice inflates performance by up to 0.137 Spearman, and Section 6 shows the most expensive feature block contributes almost nothing.

The data comprises 840 labeled mutations over 28 proteins in 7 families (PF01–PF07), 180 unlabeled candidates over 35 proteins in 9 families, and a 256-dimensional embedding for each of 36 proteins. Two structural facts drive every subsequent design decision. **Mutations are nested within proteins, and proteins within families** — 840 mutations cover only 28 proteins, so rows are not independent. And **the candidate set is partly out-of-distribution by construction**: 58 of 180 candidates (32%) belong to families PF08 and PF09, which have no labeled examples at all. Any validation scheme ignoring the first will overstate performance; any prioritization ignoring thesecond will present extrapolations as interpolations.

2. Data and preprocessing

Implemented in `src/data_processing.py` .

Mutation tables join to the embedding table on `protein_id` (many-to-one). The join is verified, not assumed: cardinality is asserted via `validate="many_to_one"` ; zero row loss is enforced
(840 → 840, 180 → 180); vector alignment is re-checked element-wise after the merge, which catches a misaligned join that preserved row counts; and the two columns appearing in both tables are confirmed to agree per protein before one copy is dropped.

The audit scans for nulls, infinities and range violations. **The dataset is clean: zero missing, zero infinite, zero out-of-range values** in both tables. The handling policy is nonetheless
implemented and tested, because a clean dataset today is no guarantee for a refresh: a missing *target* drops the row rather than fabricating supervision, a missing *feature* takes the median fitted on labeled data only, and out-of-range values are logged but **never silently clipped**.

Feature engineering. Secondary structure (H/E/C) is one-hot encoded against a fixed category list, so training and candidate matrices always share identical columns in identical order.
Physicochemical deltas capture the shift the substitution imposes:

$$\Delta_{\text{property}} = \text{property}_{\text{mutant}} - \text{property}_{\text{wild-type}}$$

for hydrophobicity, formal charge and residue volume, in both signed and absolute form. The signed delta carries *direction* — a hydrophobic-to-polar change behaves very differently in the core than
on the surface — while the absolute delta carries *magnitude of perturbation*, which a tree model would otherwise need two splits to express.
Relative position (`position / protein_length`)
makes sequence location comparable across proteins of different length.

Output: **22 tabular + 256 embedding features**, recorded in the manifest so no downstream stage re-derives them.

3. Modeling approach

Each target is modelled by independent single-target regression. Multi-output regression was rejected deliberately: the two targets have different scales, different error structure and — as Section 5 shows — different achievable accuracy, and nothing requires a shared representation.

Classical pipeline (`src/model.py` , `src/train.py`). Every fold fits one scikit-learn `Pipeline` from scratch:

```
ColumnTransformer
  ├─ tabular   (22 cols) : SimpleImputer(median) → StandardScaler
  └─ embedding (256 cols): SimpleImputer(median) → StandardScaler → PCA(16)
→ GradientBoostingRegressor
```

Because imputation, scaling and PCA are **steps inside the pipeline**, `fit` on the training fold is the only place their statistics are learned. This is the central leakage guarantee, and it is asserted rather than assumed: multiplying validation features by 1000 while setting validation targets to –999 leaves every fitted transformer and model coefficient bit-identical. PCA is fitted per fold, never globally.

Neural architectures (`src/torch_model.py` , `src/torch_model_v2.py`). Two were implemented on a shared `FoldTrainer` abstraction that owns the epoch loop, early stopping and target scaling:

v1, multitask MLP is a shared trunk over all 278 features with two task heads. **v2, dual-branch** adds a tabular branch, a separate embedding branch compressed through a bottleneck, residual fusion, then two heads — a separation responding directly to the Section 6 finding that the embedding is per-protein-constant, preventing 256 uninformative dimensions from swamping 22 informative ones.

Neural training has **two** leakage boundaries, not one: the inner validation split used for early stopping is carved from the training fold and grouped like the outer split, so no protein informs its own stopping epoch.

Model selection used pooled out-of-fold Spearman under `family_group` , chosen because the deliverable is a *ranking*: Spearman measures the ordering quality Top-10 selection depends on, while RMSE would reward calibration the downstream task never uses. Pearson is reported alongside so poor calibration stays visible, since a model can order candidates perfectly while systematically mis-scaling magnitudes.

4. Validation strategy

Implemented in `src/validation.py` .

Let mutation i belong to protein $p(i)$ and family $f(p(i))$. Standard K-fold partitions the 840 mutations at random, so for almost every fold there exist $i \in \text{train}$ and $j \in \text{validation}$ with $p(i) = p(j)$. The model can then score well on j by recognising the protein rather than learning the substitution's effect — and because the protein embedding is **constant within a protein**, it is a perfect protein identifier. Three

nested regimes were compared: `random` (optimistic reference), `protein_group` (transfer to an unseen protein in a *seen* family) and `family_group` (transfer to an unseen *family* — the candidates' regime).

Every fold set is verified before use: each row validated exactly once, no row on both sides, and no group shared across a grouped split. A violation raises `LeakageError` rather than producing a quietly optimistic number. Measured on the labeled set:

Strategy	Val rows/fold	Proteins on both sides	Families on both sides
`random`	168 (exactly)	**28 of 28**	**7 of 7**
`protein_group`	155–183	0	up to 5
`family_group`	106–246	0	0

Random K-fold places **every** protein and **every** family on both sides of **every** fold. The leakage is not marginal; it is total.

The optimism gap. Pooled out-of-fold Spearman for the selected gradient-boosted model:

Target	`random`	`protein_group`	`family_group`	Gap (random – family)
`stability_ddG`	0.650	0.656	**0.556**	**+0.094**
`functional_score`	0.817	0.771	**0.747**	**+0.070**

Across all models the gap reaches **+0.137** (ridge, stability) — a random split would have reported ridge at 0.644, competitive with gradient boosting's 0.650, when its true out-of-family performance is 0.507. Two readings follow. **Generalising to a new protein within a known family is nearly free** (`protein_group` ≈ `random` , 0.656 vs 0.650): family-level information is what the model relies on. **Generalising to a new family is where performance is actually lost** — and that is precisely the regime of 32% of the candidates.

`family_group` is therefore the **primary** strategy for model selection, ablation and every headline figure. Because its folds are uneven (106–246 rows, a 2.3× imbalance), all headline metrics are **pooled across out-of-fold predictions** rather than averaged over folds: pooling weights every mutation equally and penalises differences in baseline level between families, which is what ranking candidates across families demands. Per-fold standard deviations are reported alongside (`fold_std_spearman` ≈ 0.03–0.06).

`results/figures/strategy_comparison.png` plots this across all models: bars fall from left to right within every model as the split grows stricter, and that fall is the optimism a random split would have bought.

5. Results

Pooled out-of-fold under `family_group` , all models:

Model	`stability_ddG` ρ	Pearson	`functional_score` ρ	Pearson
Ridge (linear)	0.507	0.513	0.708	0.730
Random forest	0.522	0.548	0.721	0.776
Gradient boosting	**0.556**	**0.598**	**0.747**	**0.796**
Multitask MLP	0.551	0.531	0.218	0.222
Dual-branch NN	0.390	0.379	0.282	0.301

Gradient boosting wins both targets, and the neural networks do not beat it. On stability the multitask MLP is competitive (0.551 vs 0.556); on function both networks fail badly, with $R^2 \leq 0$ — worse than predicting the training mean. 840 rows across 7 families is too little data for these architectures, so gradient boosting is the model to ship. The networks are retained because the comparison is itself a result: the dual-branch architecture was motivated by a correct diagnosis (Section 6) yet still underperformed, and a controlled 2×2 showed its architecture and loss-weight changes interact rather than contribute independently.

Selected model, full metrics:

Target	MAE	RMSE	R^2	Pearson	Spearman	Fold SD (ρ)
`stability_ddG`	0.338	0.426	0.341	0.598	0.556	0.052
`functional_score`	0.055	0.069	0.634	0.796	0.747	0.031

Functional retention is substantially more predictable than stability (R^2 0.63 vs 0.34). The consequence for reading the Top 10 is important: with RMSE 0.426 on a target whose predicted range
is –0.149 to +1.41, the model resolves *coarse* stability ordering but cannot distinguish two candidates differing by less than roughly 0.4 kcal/mol.

Fit and residual plots are in `results/figures/pred_vs_actual.png` and `residual_distribution.png` : predicted `stability_ddG` is visibly compressed relative to its observed range — the visual form of $R^2 = 0.34$ — and both targets are near-unbiased in the mean, with stability carrying the heavier residual tail.

Reproducibility is pinned by passing the seed explicitly to every stochastic component. Two runs in one environment agree to floating-point tolerance (maximum out-of-fold difference 6.7e-16);
they are not byte-identical, because threaded BLAS does not fix summation order. Metrics at reported precision, the selected models and the Top 10 are unchanged between runs.

6. Ablation and interpretation

Ablation (`src/ablation.py`). Each subset is cross-validated from scratch under `family_group` with folds, seed and model fixed, so differences are attributable to features alone. The five groups are `physicochemical` (6 features), `structural` (7), `sequence` (3), `deltas` (6) and `embeddings` (256).

Subset	Features	`stability_ddG`	Δ	`functional_score`	Δ
D_full	22 + 256	**0.556**	—	**0.747**	—
E_all_tabular_no_embeddings	22 + 0	0.546	–0.010	0.723	–0.025
F_full_minus_deltas	16 + 256	0.536	–0.021	0.717	–0.030
G_full_minus_structural	15 + 256	0.413	–0.143	0.370	–0.377
C_tabular_plus_embeddings	9 + 256	0.397	–0.159	0.327	–0.421
A_tabular_no_embeddings	9 + 0	0.373	–0.183	0.271	–0.477
B_embeddings_only	0 + 256	**–0.205**	–0.762	**–0.110**	–0.857

(1) The structural microenvironment carries almost all the signal. Removing `structural` costs –0.143 and –0.377 — by far the largest effect. Conservation, solvent accessibility and distance to active site describe the local constraint the substitution must satisfy, which is biophysically what one would expect to matter.

(2) The 256-dimensional embeddings contribute essentially nothing, and alone they are worse than useless. Adding them to the full tabular set buys +0.010 and +0.025 Spearman; used alone they produce *negative* rank correlation. The reason is structural, not a bug: the embedding is a **per-protein constant**, so it cannot discriminate between mutations in the same protein — the actual task — and can only shift a family-level baseline, which under family-grouped CV is unseen and therefore generalises poorly. **Subset E (22 features, no embeddings, no PCA) performs within 0.01–0.03 Spearman of the full 278-feature model** at a fraction of the cost.

(3) Engineered deltas earn their place, modestly — removing them costs –0.021 and –0.030, small but consistent, and larger than the embeddings' contribution.

Feature attribution (`src/attribution.py`). Permutation importance is computed on *held-out* rows and shuffled **before** the pipeline, so importance is defined on the original named columns; the 256 embedding columns are shuffled as **one unit**, since shuffling correlated columns individually would understate a block whose members substitute for each other. Impurity importance is reported alongside with its biases stated. Mean over the 5 folds:

Target	Top features (permutation, share of total)
`stability_ddG`	`solvent_accessibility` 0.108 (23%), `delta_hydrophobicity` 0.107 (23%), `mutant_hydrophobicity` 0.092 (20%)
`functional_score`	`distance_to_active_site` 0.314 (**49%**), `abs_delta_charge` 0.174 (27%), `abs_delta_hydrophobicity` 0.067 (10%)

The embedding block's permutation importance is –0.0020 (stability) and +0.0004 (function) — indistinguishable from zero. Two independent methods, ablation and permutation, converge on the same verdict; that agreement is the strongest interpretation result here. Note also that the two targets are driven differently: function by *where* the mutation sits relative to the active site, stability by *what* the substitution changes physicochemically. They should not share a single ranking signal uncritically.

7. Mutation prioritization

Implemented in `src/predict.py` .

Ensemble inference. The five `family_group` fold pipelines per target form an ensemble, giving a mean prediction $\hat{y}_t(c)$ and the spread $s_t(c)$ (sample standard deviation across the $K = 5$ fold models). Each fold model carries its own fitted transformers, so none is reused.

Viability gate, then score. A candidate is *viable* only if predicted ddG ≤ 0 **and** functional score ≥ 0.7. This hard constraint sorts **before** the soft score, so no weighting can promote a non-viable candidate above a viable one. The soft score is then

$$\text{priority}(c) = w_{\text{stab}}\,\widetilde{(-\Delta\Delta G)}(c) + w_{\text{func}}\,\widetilde{S_{\text{func}}}(c) - \lambda\,\tilde{u}(c), \qquad u(c) = \sqrt{s^2_{\text{stab}}(c) + s^2_{\text{func}}(c)}$$

where $\tilde{\cdot}$ is percentile rank within the 180 candidates, $w_{\text{stab}} = w_{\text{func}} = 0.5$ and $\lambda = 0.25$. Uncertainties combine in quadrature as independent sources, and the penalty applies to the *normalised* uncertainty so it is commensurate with the objective terms. Ties break on `mutation_id` , so the ranking is reproducible.

Distribution-shift flagging, and a negative result that matters. 58 of 180 candidates come from the unseen families PF08/PF09. Ensemble spread does **not** detect this shift: mean stability spread is **0.0969** for the 58 unseen-family candidates against **0.1031** for the 122 in-distribution ones, and mean function spread **0.0146** against **0.0160**.

Fold disagreement is *slightly lower* for out-of-distribution candidates. The five fold models agree with one another on novel families while potentially all being wrong together — exactly the shared error a disagreement-based estimate cannot see. **The uncertainty penalty therefore does not protect against domain shift; only the explicit `ood_flag` does.** Ensemble spread is a lower bound on predictive uncertainty, capturing neither label noise nor error common to every fold.

7.1 Final Top 10

Substitutions are given as wild-type/position/mutant; SS is the secondary structure at that position; full rows including protein id are in `results/mutation_rankings/top_10_candidates.csv` .

Rank	Mutation	Fam.	Sub.	SS	ddG ± s	Func ± s	Score	Viability
1	CAND0148	PF06	S490L	H	–0.013 ± 0.085	0.861 ± 0.027	0.850	pass
2	CAND0162	PF03	Y12F	C	**–0.149** ± 0.038	0.800 ± 0.013	0.833	pass
3	CAND0167	PF03	N133S	C	–0.068 ± 0.125	0.854 ± 0.013	0.750	pass

Rank	Mutation	Fam.	Sub.	SS	ddG ± s	Func ± s	Score	Viability
4	CAND0015	PF03	E353V	C	−0.076 ± 0.089	0.750 ± 0.014	0.631	pass
5	CAND0126	PF06	Q360F	C	−0.019 ± 0.156	0.798 ± 0.012	0.619	pass
6	CAND0002	PF02	M496W	E	+0.234 ± 0.040	0.878 ± 0.013	**0.885**	fails_stability
7	CAND0034	PF01	W11Y	C	+0.252 ± 0.020	0.865 ± 0.014	0.865	fails_stability
8	CAND0121	PF02	I66V	H	+0.172 ± 0.078	0.885 ± 0.006	0.846	fails_stability
9	CAND0159	PF01	C14V	E	+0.111 ± 0.070	0.851 ± 0.006	0.842	fails_stability
10	CAND0175	PF05	C475M	C	+0.263 ± 0.060	0.905 ± 0.007	0.835	fails_stability

Three observations:

- **Only 5 of 180 candidates clear the stability gate.** The gate is therefore the binding constraint, not the score: ranks 6–10 are reported *and labelled* `fails_stability` rather than excluded, so the shortlist stays the requested length while the gate remains visible.
  Rank 6 (CAND0002) carries the **highest priority score in the entire pool** (0.885) yet ranks sixth — the constraint overriding the score, visible in one row.
- **No out-of-distribution candidate reached the Top 10.** All ten are from represented families despite 32% of the pool being unseen-family. This is favourable for reliability, but it is an emergent property of the scoring, not an enforced constraint; exploring novel families requires requesting the flagged candidates explicitly.
- **Predicted functional retention spans only 0.750–0.905 across the Top 10**, narrower than the pooled RMSE for that target would nominally resolve (0.069). Ordering *within* the Top 10 should not be read at single-rank granularity; the defensible claim is that these ten are collectively better than the rest of the pool.

7.2 Scientific interpretation of selected candidates

Generated by `src/interpret.py` ; full document in `results/mutation_rankings/candidate_interpretation.md` . Qualitative bands are the pool's 33%/67% quantiles. Three candidates are discussed because they span deliberately different scientific situations rather than simply the top three.

Rank 2 — CAND0162 (PF03): Y12F — the only resolved prediction. Tyrosine → phenylalanine in a coil. ddG **−0.149 ± 0.038**, function **0.800 ± 0.013**. This is the **only shortlisted candidate whose stabilizing prediction is resolved**: the magnitude is 3.9× its own spread, against 0.12–0.85 for the other four viable entries. Conservation **0.186** (lower third), solvent accessibility **0.940** (almost fully exposed), Δvolume **−3.7 Å³** (near-isosteric), Δhydrophobicity **+4.1**.
Y→F removes one hydroxyl while preserving the aromatic ring, so the near-zero Δvolume is consistent; at a highly exposed position, losing the polar hydroxyl raises hydrophobicity without disturbing packing. The cost is function: 0.800 is the second-lowest in the Top 10. **On a stability-weighted campaign this would be the lead candidate** — changing `prioritization.weights` in config reorders the list without touching code.

Rank 1 — CAND0148 (PF06): S490L — ranked first, sign unresolved. Serine → leucine in a helix.
ddG **−0.013 ± 0.085**, function **0.861 ± 0.027**. The ddG magnitude is roughly **six times smaller than its own spread**, so the honest statement is "approximately neutral", not "stabilizing". Conservation 0.333, solvent accessibility 0.698, with a polar→hydrophobic substitution (Δhydrophobicity **+4.6**) and a large volume increase (**+77.7 Å³**). On standard biophysical grounds, burying a large hydrophobe is stabilizing in a core but entropically unfavourable on an **exposed helix face**, so mild destabilization would be the more expected outcome — the physicochemistry argues *against* the model here. It earns rank 1 on strong predicted function plus a stability estimate that is merely not-bad, and it is the entry most in need of measurement rather than trust.

Rank 14 — CAND0091 (PF08): Y69W — where confidence is least deserved. The best-ranked *unseen-family* candidate, with function **0.870 ± 0.004** — the **tightest uncertainty of any candidate discussed**, on the one mutation where the model has least right to be confident. This makes the Section 7 negative result concrete: ensemble spread measures whether the five fold models *agree*, not whether they are jointly wrong, and on a novel family they agree precisely because they share a blind spot. Only the explicit `ood_flag` protects against this — never the uncertainty number.

Cross-cutting. The viable candidates share **low-to-moderate conservation** and **moderate-to-high solvent exposure** (0.57–0.94, against a pool median near 0.44). The model has learned a recognisable rule — *perturb loosely constrained, exposed positions* — which is the heuristic an engineer would apply manually, and which Section 6 traced almost entirely to the structural features rather than the pretrained embeddings. One caution: **Δcharge is 0.0 for every candidate discussed**, so the top picks are all charge-neutral. Whether that reflects a learned preference or simply that charge-changing substitutions are rarer in this pool is not determinable from these cases.

8. Limitations

- **Synthetic data.** Every number above describes a synthetic dataset — declared as such by its own `README_DATASET.txt`, with both targets labelled synthetic field by field. The data's own fingerprints corroborate it: the embeddings are standard-normal noise (mean −0.004, sd 0.886) and amino-acid usage is uniform (3.8–6.0% across all 20 residues, against a real proteome's sharp skew). The relationships are generator artefacts and **no biophysical conclusion transfers to real proteins.** What transfers is the *methodology*: the leakage control, the ablation design and the uncertainty audit.
- **7 labeled families is too few for stable family-grouped CV.** Folds hold 1–2 families and vary 2.3× in size. Pooled metrics mitigate this but do not remove it; a ±0.05 Spearman difference between models should not be over-interpreted.
- **Stability is only coarsely predicted** (R^2 0.34, RMSE 0.426), so differences below ~0.4 kcal/mol are not resolved — which directly limits how finely the Top 10 can be read.
- **Uncertainty is underestimated.** Ensemble spread is blind to domain shift (Section 7); it quantifies model disagreement, not total predictive error.
- **Scores are pool-relative.** Rank normalisation is internal to these 180 candidates, so `priority_score` is not an absolute measure of merit; `is_viable` is.
- **Attribution is global, not per-prediction.** Per-candidate attributions (SHAP) would let the Section 7.2 narratives cite model-grounded contributions rather than input values plus general biophysics — the cheapest available improvement.
- **Results are environment-sensitive.** A scikit-learn version change shifts predictions slightly and can reorder the Top 10; pinned versions and the seed are in `results/models/training_metadata.json`.

9. Future research direction

Replace the per-protein embedding with a mutation-aware representation — highest value, given the ablation. The embedding's failure is entirely explained by its being constant within a protein. Per-*residue* representations from an ESM-style protein language model give a position-specific vector, and the standard zero-shot signals — wild-type-vs-mutant log-odds at the mutated position, and pseudo-log-likelihood difference — are strong precisely because they vary per mutation. This attacks the bottleneck both ablation and permutation independently identified, rather than adding capacity elsewhere. The ablation harness should be re-run before anything larger is trained.

Structure-aware modelling — the most promising axis, since the structural block dominates both analyses. With AlphaFold or ESMFold structures, the hand-crafted structural features could be replaced by learned ones: a graph neural network over the residue contact graph with the mutated residue as query node, then an SE(3)-equivariant architecture, capturing the packing that `solvent_accessibility` and `distance_to_active_site` only approximate. The dual-branch design is
already the right shape — the embedding branch is swapped for a structural encoder. Note this needs data absent here: message passing requires edges encoding real physics, and this dataset supplies no coordinates, sequences or contact map (`distance_to_active_site` is a distance to a *single* point, from which pairwise distances are not recoverable), so any graph built now would have fabricated edges. Fusion should carry per-task loss scaling from the start — the v2 result showed that separating the branches was necessary but *not sufficient* until the loss weighting was fixed.

Then, in order: MSA-derived PSSMs and Potts/DCA couplings, carrying MSA depth as a feature; MD-derived RMSF, SASA variance and local unfolding propensity, aimed at the weaker stability target; and calibrated uncertainty — deep ensembles, evidential regression or conformal prediction, plus a continuous distance-to-training-data score replacing the binary flag — **before** any active learning, since an acquisition function blind to novelty would under-explore exactly the novel families a campaign most needs. Generative design belongs last: a generator is only as good as its oracle, and this oracle cannot resolve 0.4 kcal/mol. With more families, leave-one-family-out and nested CV become

practical, the latter allowing hyperparameter tuning without the optimistic bias that tuning on the selection metric introduces. The full nine-step plan is in `README.md`

What should not change. The grouped CV, the leakage assertions, the ablation harness and config-driven reproducibility are not scaffolding to discard once better models arrive — they are what would make a better model's improvement *believable*.

Appendix: reproducing these results

After `pip install -r requirements-dev.txt`, `python main.py --config config/config.yaml` reproduces every number above in ~126 s (807 tests pass via `python -m pytest -q`). Ten stages run in dependency order — `data_processing → validation → train → train_torch → train_torch_v2 → evaluate → predict → ablation → attribution → interpret` — each also runnable alone as `python -m src.<stage>`. A per-task walkthrough, with the command and outputs for each, is in `report/task_wise_report.md`. Requires Python ≥ 3.11; all configuration lives in `config/config.yaml`.